

核心代码

智驾未来 · AI面试仓

上海第九届青少年AI创新大赛 / 智能面试仓的开发与应用

功能选编 · 统一提交 · 可追溯行号

源码基线 b352fbbb25eb2dcaed39f0e7900be5b08fcfcbad

编制日期 2026-09-30

阅读方式 每段标注仓库路径及原文件行号；长行视觉软换行，原文件行号不变。

材料范围 精选当前功能的关键实现；不是完整源码或模型效果证明。

目录与使用说明

所有摘录均由同一 Git 提交读取；页脚短 SHA 对应封面完整 SHA。

01 数据骨架与身份	03
02 六题装配与开场	05
03 服务端状态与追问	08
04 答题并发与落库	11
05 ASR 与可复现声学规则	16
06 评分证据与模型回退	19
07 成长记录与曲线	24
08 企业同源初筛	29
09 新生咨询与学习	33
10 逐题反馈隔离	37
答辩说明与实现边界	39

目录页码对应页脚编号。

01 数据骨架与身份

证明学生、岗位、答题、报告的关系和访问边界均由服务端保存。

学生档案

server/models.py / 行 6-24

```
6 class User(Base):
7     __tablename__ = "users"
8     __table_args__ = (Index("ux_users_role_login_email", "role", "login_email", unique=True),)
9     id: Mapped[int] = mapped_column(Integer, primary_key=True)
10    role: Mapped[str] = mapped_column(String(32))
11    name_masked: Mapped[str|None] = mapped_column(String(128), nullable=True)
12    major: Mapped[str|None] = mapped_column(String(128), nullable=True)
13    grade: Mapped[str|None] = mapped_column(String(64), nullable=True)
14    student_no: Mapped[str|None] = mapped_column(String(64), nullable=True, unique=True)
15    education_level: Mapped[str|None] = mapped_column(String(32), nullable=True)
16    school_tier: Mapped[str|None] = mapped_column(String(32), nullable=True)
17    internship_experience: Mapped[str|None] = mapped_column(Text, nullable=True)
18    awards: Mapped[str|None] = mapped_column(Text, nullable=True)
19    resume_storage_key: Mapped[str|None] = mapped_column(String(80), nullable=True)
20    resume_original_name: Mapped[str|None] = mapped_column(String(255), nullable=True)
21    login_email: Mapped[str|None] = mapped_column(String(254), nullable=True)
22    password_hash: Mapped[str|None] = mapped_column(String(255), nullable=True)
23    organization_name: Mapped[str|None] = mapped_column(String(128), nullable=True)
24    contact_name: Mapped[str|None] = mapped_column(String(128), nullable=True)
```

会话快照与租约字段

server/models.py / 行 43-50

```
43 class Session(Base):
44     __tablename__ = "sessions"
45     id: Mapped[int] = mapped_column(Integer, primary_key=True); user_id: Mapped[int] = mapped_column(Integer); job_id: Mapped[
46         int] = mapped_column(Integer)
47     mode: Mapped[str] = mapped_column(String(32)); started_at: Mapped[datetime|None] = mapped_column(DateTime, nullable=True)
48     ; status: Mapped[str] = mapped_column(String(32))
49     pending_question_json: Mapped[dict|None] = mapped_column(JSON, nullable=True)
50     lease_token: Mapped[str|None] = mapped_column(String(64), nullable=True)
51     lease_expires_at: Mapped[datetime|None] = mapped_column(DateTime, nullable=True)
52     input_mode: Mapped[str|None] = mapped_column(String(16), nullable=True)
```

回答与报告

server/models.py / 行 51-55

```
51 class Answer(Base):
52     __tablename__ = "answers"
53     id: Mapped[int] = mapped_column(Integer, primary_key=True); session_id: Mapped[int] = mapped_column(Integer); q_seq: Mapped[
54         int] = mapped_column(Integer)
55     question_text: Mapped[str] = mapped_column(Text); answer_text: Mapped[str] = mapped_column(Text); is_followup: Mapped[boo
56         l] = mapped_column(Boolean)
57     is_retry: Mapped[bool] = mapped_column(Boolean); duration_s: Mapped[float|None] = mapped_column(Float, nullable=True); wp
58         m: Mapped[float|None] = mapped_column(Float, nullable=True); pause_cnt: Mapped[int|None] = mapped_column(Integer, nullable=Tr
59         ue); filler_cnt: Mapped[int|None] = mapped_column(Integer, nullable=True)
```

报告记录

server/models.py / 行 56-60

```
56 class Report(Base):
57     __tablename__ = "reports"
58     id: Mapped[int] = mapped_column(Integer, primary_key=True); session_id: Mapped[int] = mapped_column(Integer)
59     dimensions_json: Mapped[dict|None] = mapped_column(JSON); highlights_json: Mapped[dict|None] = mapped_column(JSON); conce
rns_json: Mapped[dict|None] = mapped_column(JSON); improvement_json: Mapped[dict|None] = mapped_column(JSON); overall: Mapped
[float|None] = mapped_column(Float, nullable=True)
60     scoring_version: Mapped[str|None] = mapped_column(String(16), nullable=True)
```

会话鉴权

server/api/auth.py / 行 112-124

```
112 def require_current_user(request: Request, db: Session = Depends(get_db)) -> User | None:
113     if not settings.auth_required:
114         return None
115     token = request.cookies.get(AUTH_COOKIE_NAME)
116     user = auth_service.resolve_auth_session(db, token)
117     if user is None:
118         raise HTTPException(
119             status_code=401,
120             detail={"code": "AUTH_REQUIRED", "message": "请先登录"},
121             headers={"WWW-Authenticate": "Cookie"},
122         )
123     request.state.current_user = user
124     return user
```

学生角色门禁

server/api/auth.py / 行 127-130

```
127 def require_student(user: User | None = Depends(require_current_user)) -> User | None:
128     if user is not None and user.role != "student":
129         raise HTTPException(status_code=403, detail={"code": "ROLE_FORBIDDEN", "message": "仅学生账号可访问"})
130     return user
```

会话路由角色依赖 [独立逻辑块]

server/api/sessions.py / 行 40-40

```
40 router = APIRouter(prefix="/api/sessions", tags=["sessions"], dependencies=[Depends(require_student)])
```

会话本人访问校验

server/api/sessions.py / 行 43-55

```
43 def _ensure_student_session_access(request: Request, sid: int) -> None:
44     if not settings.auth_required:
45         return
46     current_user = getattr(request.state, "current_user", None)
47     if current_user is None:
48         raise HTTPException(status_code=401, detail={"code": "AUTH_REQUIRED", "message": "请先登录"})
49     with SessionLocal() as db:
50         owned = db.query(Session.id).filter(Session.id == sid, Session.user_id == current_user.id).first()
51     if owned is None:
52         raise HTTPException(
53             status_code=404,
54             detail={"code": "SESSION_NOT_FOUND", "message": "面试会话不存在"},
55         )
```

02 六题装配与开场

证明题库按两道通用、三道专业、一道情景装配，缺题拒绝创建；开场并行预取语音。

题型数量与缺题异常 [独立逻辑块]

server/services/question_bank.py / 行 6-10

```
6 INTERVIEW_QUESTION_COUNTS = {"通用": 2, "专业": 3, "情景": 1}
7
8
9 class QuestionBankIncompleteError(ValueError):
10     pass
```

题库完整性

server/services/question_bank.py / 行 13-32

```
13 def get_interview_questions(db: Session, job_id: int) -> list[Question]:
14     """Pick the most recently maintained questions while preserving the 2/3/1 flow."""
15     selected: list[Question] = []
16     missing: dict[str, int] = {}
17     for question_type, required_count in INTERVIEW_QUESTION_COUNTS.items():
18         newest = (
19             db.query(Question)
20             .filter(Question.job_id == job_id, Question.type == question_type)
21             .order_by(Question.id.desc())
22             .limit(required_count)
23             .all()
24         )
25         if len(newest) < required_count:
26             missing[question_type] = required_count - len(newest)
27         selected.extend(reversed(newest))
28
29     if missing:
30         details = ", ".join(f"{kind}缺少{count}题" for kind, count in missing.items())
31         raise QuestionBankIncompleteError(details)
32     return selected
```

创建会话与并行预取

server/api/sessions.py / 行 115-193

```
115 async def create_session(req: SessionCreateRequest, request: Request):
116     with SessionLocal() as db:
117         job = db.query(Job).filter(Job.id == req.job_id).first()
118         if not job:
119             raise HTTPException(
120                 status_code=404,
121                 detail={"code": "JOB_NOT_FOUND", "message": "岗位不存在"},
122             )
123
124         user = db.query(User).filter(User.id == req.user_id).first()
125         if not user or user.role != "student":
126             raise HTTPException(
127                 status_code=404,
128                 detail={"code": "USER_NOT_FOUND", "message": "学生档案不存在"},
129             )
130         current_user = getattr(request.state, "current_user", None)
131         if current_user is not None and current_user.id != user.id:
132             raise HTTPException(
133                 status_code=404,
134                 detail={"code": "USER_NOT_FOUND", "message": "学生档案不存在"},
135             )
136
137         try:
138             questions = get_interview_questions(db, req.job_id)
```

创建会话与并行预取 / 续 / server/api/sessions.py

```

139         except QuestionBankIncompleteError as exc:
140             raise HTTPException(
141                 status_code=409,
142                 detail={"code": "QUESTION_BANK_INCOMPLETE", "message": str(exc)},
143             ) from exc
144
145         q1_text = questions[0].text
146         question_texts = [q.text for q in questions[:6]]
147         pending_q = {
148             "seq": 1,
149             "text": q1_text,
150             "audio_url": None,
151             "is_followup": False,
152         }
153
154         session = Session(
155             user_id=req.user_id,
156             job_id=req.job_id,
157             mode=req.mode,
158             started_at=datetime.utcnow(),
159             status="active",
160             pending_question_json=pending_q,
161             lease_token=None,
162             lease_expires_at=None,
163             input_mode=None,
164         )
165         db.add(session)
166         db.commit()
167         db.refresh(session)
168         sid = session.id
169
170         # 在会话创建后并行预取固定题和过渡语音频
171         prefetch_results, transition_urls = await asyncio.gather(
172             tts.prefetch_session_questions(sid, question_texts),
173             tts.prefetch_session_transitions(sid),
174         )
175         q1_audio_url = prefetch_results.get(1) if isinstance(prefetch_results, dict) else None
176
177         # 若首题预取成功, 更新持久化待答状态中的 audio_url
178         if q1_audio_url:
179             with SessionLocal() as db_update:
180                 sess_obj = db_update.query(Session).filter(Session.id == sid).first()
181                 if sess_obj and sess_obj.pending_question_json:
182                     pending_updated = dict(sess_obj.pending_question_json)
183                     pending_updated["audio_url"] = q1_audio_url
184                     sess_obj.pending_question_json = pending_updated
185                     db_update.commit()
186
187         valid_trans_urls = [u for u in transition_urls if u is not None] if isinstance(transition_urls, list) else []
188
189         return SessionCreateResponse(
190             sid=sid,
191             question=QuestionResponse(text=q1_text, audio_url=q1_audio_url, seq=1),
192             transition_audio_urls=valid_trans_urls,
193         )

```

问题语音预取

server/services/tts.py / 行 83-123

```

83 async def prefetch_session_questions(session_id: int, questions: list) -> dict[int, str | None]:
84     """会话创建时使用 asyncio.gather 并行预取全部固定主问题的 MP3 音频。
85     单条失败不阻塞其他题目与会话创建, 每条异常单独捕获并记为 None。
86     """
87     audio_dir = Path(settings.tts_output_dir)

```

问题语音预取 / 续 / server/services/tts.py

```
88     results: dict[int, str | None] = {}
89
90     async def _synth_single(seq: int, text: str) -> tuple[int, str | None]:
91         if not settings.tts_enabled:
92             return seq, None
93         filename = f"session_{session_id}_q{seq}.mp3"
94         file_path = audio_dir / filename
95
96         if not _discard_stale_audio(file_path):
97             return seq, None
98
99         try:
100             ok = await synthesize_to_file(text, file_path)
101             if ok:
102                 return seq, get_audio_url_if_exists(filename)
103         except Exception as exc:
104             logger.warning("会话 %s 预取题目 %s 异常: %s", session_id, seq, exc)
105         return seq, None
106
107     tasks = []
108     for seq, item in enumerate(questions, 1):
109         text = item if isinstance(item, str) else getattr(item, "text", str(item))
110         tasks.append(_synth_single(seq, text))
111
112     if not tasks:
113         return results
114
115     gathered = await asyncio.gather(*tasks, return_exceptions=True)
116     for item in gathered:
117         if isinstance(item, tuple) and len(item) == 2:
118             seq, url = item
119             results[seq] = url
120         elif isinstance(item, Exception):
121             logger.warning("预取任务异常: %s", item)
122
123     return results
```

03 服务端状态与追问

证明待答题快照可恢复，追问受提示词和每题一次的服务端判断约束。

状态恢复

server/api/sessions.py / 行 59-85

```
59 def get_session_state(request: Request, sid: int = Path(..., ge=1)):
60     _ensure_student_session_access(request, sid)
61     # One SQL statement gives a consistent view of the committed answer and question.
62     count = select(func.count(Answer.id)).where(Answer.session_id == Session.id).correlate(Session).scalar_subquery()
63     report_id = select(func.max(Report.id)).where(Report.session_id == Session.id).correlate(Session).scalar_subquery()
64     with SessionLocal() as db:
65         row = db.execute(select(Session, count, report_id).where(Session.id == sid)).first()
66         if row is None:
67             raise HTTPException(status_code=404, detail={"code": "SESSION_NOT_FOUND", "message": "面试会话不存在"})
68         sess, answer_count, rid = row
69         pending = sess.pending_question_json or {}
70         question = None
71         if (
72             sess.status != "completed"
73             and isinstance(pending.get("text"), str)
74             and isinstance(pending.get("seq"), int)
75         ):
76             question = QuestionResponse(
77                 text=pending["text"], seq=pending["seq"], audio_url=pending.get("audio_url")
78             )
79         return SessionStateResponse(
80             sid=sid, status=sess.status, answer_count=answer_count,
81             question=question,
82             is_followup=bool(pending.get("is_followup", False)), report_id=rid,
83             user_id=sess.user_id, job_id=sess.job_id, mode=sess.mode,
84             input_mode=sess.input_mode,
85         )
```

追问判断与失败降级

server/services/orchestrator.py / 行 15-56

```
15 async def evaluate_followup(
16     question_text: str,
17     answer_text: str,
18     followup_hint: Optional[str],
19     job_info: dict,
20 ) -> Optional[str]:
21     """判断是否对当前回答发起追问（每题最多1次，提示词包含严格否定约束）。
22     所有异常降级必须静默可用：若 LLM 故障或解析失败，直接返回 None（进入下一题）。
23     """
24     system_prompt = (
25         "你是一名智能汽车产业面试考官。请根据候选人的回答，判断是否需要发起一次深入技术追问。\\n"
26         "【严格否定约束】\\n"
27         "1. 禁止评价回答好坏（严禁出现“回答得很好”、“不错”、“欠缺”等评语）；\\n"
28         "2. 禁止重复原问题；\\n"
29         "3. 追问正文严格禁止超过一句话；\\n"
30         "4. 若候选人回答已较完整，或追问价值不大，必须返回 need_followup: false。"
31     )
32     terms = job_info.get("terms") or []
33     terms_str = ", ".join(terms) if isinstance(terms, list) else str(terms)
34     user_prompt = (
35         f"【岗位名称】{job_info.get('title', '')}\\n"
36         f"【岗位JD要点】{job_info.get('jd_digest', '')}\\n"
37         f"【专业术语表】{terms_str}\\n"
38         f"【原问题】{question_text}\\n"
39         f"【追问指引】{followup_hint or '无'}\\n"
40         f"【候选人回答】{answer_text}\\n\\n"
```

追问判断与失败降级 / 续 / server/services/orchestrator.py

```

41         "请判断是否追问，并输出 JSON 格式 (need_followup: bool, question: str | null) 。"
42     )
43     messages = [
44         {"role": "system", "content": system_prompt},
45         {"role": "user", "content": user_prompt},
46     ]
47
48     try:
49         decision = await chat_json("followup", messages, FollowupDecision)
50         if decision.need_followup and decision.question and decision.question.strip():
51             return decision.question.strip()
52         return None
53     except Exception as e:
54         # 所有降级必须静默可用：追问判定失败=直接下一题
55         logger.warning("Followup LLM evaluation failed, silently advancing to next question: %s", e)
56     return None

```

原题指引、追问与六题推进 [独立逻辑块]

server/api/sessions.py / 行 311-387

```

311     async def _execute_business():
312         filler_cnt = count_filler_words(req.answer_text)
313         current_seq = pending_q["seq"]
314         is_followup = pending_q.get("is_followup", False)
315         current_q_text = pending_q["text"]
316
317         orig_q = (
318             questions[current_seq - 1]
319             if 1 <= current_seq <= len(questions)
320             else None
321         )
322         followup_hint = orig_q.followup_hint if orig_q else None
323         has_followup_answered = any(
324             a.q_seq == current_seq and a.is_followup for a in current_answers
325         )
326
327         followup_text = None
328         if not is_followup and not has_followup_answered:
329             job_info = {
330                 "title": job.title,
331                 "jd_digest": job.jd_digest,
332                 "terms": job.terms_json,
333             }
334             followup_text = await orchestrator.evaluate_followup(
335                 current_q_text, req.answer_text, followup_hint, job_info
336             )
337
338         if followup_text:
339             followup_audio = await tts.synthesize_followup(sid, current_seq, followup_text)
340             next_type = "followup"
341             next_q = {
342                 "seq": current_seq,
343                 "text": followup_text,
344                 "audio_url": followup_audio,
345                 "is_followup": True,
346             }
347             next_status = "active"
348             report_data = None
349         else:
350             if current_seq < 6:
351                 next_main_q = questions[current_seq]
352                 next_main_audio = tts.get_question_audio_url(sid, current_seq + 1, is_followup=False)
353                 next_type = "next"
354                 next_q = {

```

原题指引、追问与六题推进 [独立逻辑块] / 续 / server/api/sessions.py

```
355         "seq": current_seq + 1,
356         "text": next_main_q.text,
357         "audio_url": next_main_audio,
358         "is_followup": False,
359     }
360     next_status = "active"
361     report_data = None
362     else:
363         all_answers = [a.answer_text for a in current_answers] + [
364             req.answer_text
365         ]
366         job_info = {
367             "title": job.title,
368             "jd_digest": job.jd_digest,
369             "terms": job.terms_json,
370         }
371         report_data = await scoring.score_interview(
372             answers=all_answers, job_info=job_info, mode="text"
373         )
374         next_type = "done"
375         next_q = None
376         next_status = "completed"
377
378     return (
379         next_type,
380         next_q,
381         next_status,
382         report_data,
383         current_seq,
384         current_q.text,
385         is_followup,
386         filler_cnt,
387     )
```

04 答题并发与落库

证明文本与语音共享服务端推进规则；租约互斥，失去 token 的请求不能写入最终结果。

文本回答租约抢占 [独立逻辑块]

server/api/sessions.py / 行 203-277

```
203     my_token = uuid4().hex
204     now = datetime.utcnow()
205     expires_at = now + timedelta(seconds=settings.lease_duration_seconds)
206
207     # 事务 1: 原子抢占与租约判定
208     with SessionLocal() as db:
209         result = db.execute(
210             update(Session)
211             .where(
212                 Session.id == sid,
213                 or_(
214                     Session.status == "active",
215                     and_(
216                         Session.status == "answering",
217                         Session.lease_expires_at < now,
218                     ),
219                 ),
220             )
221             .values(
222                 status="answering",
223                 lease_token=my_token,
224                 lease_expires_at=expires_at,
225             )
226         )
227         db.commit()
228
229         if result.rowcount == 0:
230             sess = db.query(Session).filter(Session.id == sid).first()
231             if not sess:
232                 raise HTTPException(
233                     status_code=404,
234                     detail={"code": "SESSION_NOT_FOUND", "message": "面试会话不存在"},
235                 )
236             if sess.status == "completed":
237                 raise HTTPException(
238                     status_code=409,
239                     detail={"code": "SESSION_COMPLETED", "message": "面试已结束"},
240                 )
241             raise HTTPException(
242                 status_code=409,
243                 detail={
244                     "code": "ANSWER_IN_PROGRESS",
245                     "message": "当前回答正在处理中，请勿重复提交",
246                 },
247             )
248
249         sess = db.query(Session).filter(Session.id == sid).first()
250         pending_q = sess.pending_question_json
251         if not pending_q or not isinstance(pending_q, dict):
252             # 缺少待答快照（旧会话），释放锁并明确提示重新开始
253             db.execute(
254                 update(Session)
255                 .where(Session.id == sid, Session.lease_token == my_token)
256                 .values(status="active", lease_token=None, lease_expires_at=None)
257             )
258             db.commit()
259             raise HTTPException(
```

文本回答租约抢占 [独立逻辑块] / 续 / server/api/sessions.py

```

260         status_code=409,
261         detail={
262             "code": "SESSION_RESET_REQUIRED",
263             "message": "该会话缺少待答题目快照, 请重新开始新面试",
264         },
265     )
266
267     _enforce_input_mode(db, sess, sid, my_token, "text")
268
269     job = db.query(Job).filter(Job.id == sess.job_id).first()
270     questions = get_interview_questions(db, sess.job_id)
271     current_answers = (
272         db.query(Answer)
273         .filter(Answer.session_id == sid)
274         .order_by(Answer.id.asc())
275         .all()
276     )
277     answer_count_before = len(current_answers)

```

租约续期 [独立逻辑块]

server/api/sessions.py / 行 284-309

```

284     async def _heartbeat_renewal():
285         while not cancel_event.is_set():
286             try:
287                 await asyncio.sleep(settings.lease_renew_interval_seconds)
288             except asyncio.CancelledError:
289                 break
290             try:
291                 with SessionLocal() as db_inner:
292                     res_renew = db_inner.execute(
293                         update(Session)
294                         .where(Session.id == sid, Session.lease_token == my_token)
295                         .values(
296                             lease_expires_at=datetime.utcnow()
297                             + timedelta(seconds=settings.lease_duration_seconds)
298                         )
299                     )
300                     db_inner.commit()
301                     if res_renew.rowcount == 0:
302                         cancel_event.set()
303                         break
304             except Exception as exc:
305                 logger.warning(
306                     "lease renewal failed for session %s: %s", sid, exc
307                 )
308
309     heartbeat_task = asyncio.create_task(_heartbeat_renewal())

```

失去租约时取消业务 [独立逻辑块]

server/api/sessions.py / 行 398-411

```
398         if cancel_task in done:
399             # 失去租约所有权! 立即取消业务任务并等待结束
400             biz_task.cancel()
401             try:
402                 await biz_task
403             except (asyncio.CancelledError, Exception):
404                 pass
405             raise HTTPException(
406                 status_code=409,
407                 detail={
408                     "code": "ANSWER_IN_PROGRESS",
409                     "message": "答题租约已被后续请求接管",
410                 },
411             )
```

文本最终 token 条件提交 [独立逻辑块]

server/api/sessions.py / 行 472-524

```
472     # 事务 3: 所有权校验与原子落库 (含首次 input_mode=text)
473     with SessionLocal() as db:
474         res = db.execute(
475             update(Session)
476             .where(Session.id == sid, Session.lease_token == my_token)
477             .values(
478                 status=next_status,
479                 pending_question_json=next_q,
480                 lease_token=None,
481                 lease_expires_at=None,
482                 input_mode=func.coalesce(Session.input_mode, "text"),
483             )
484         )
485         if res.rowcount == 0:
486             db.rollback()
487             raise HTTPException(
488                 status_code=409,
489                 detail={
490                     "code": "ANSWER_IN_PROGRESS",
491                     "message": "答题租约已被后续请求接管",
492                 },
493             )
494
495         new_answer = Answer(
496             session_id=sid,
497             q_seq=current_seq,
498             question_text=current_q_text,
499             answer_text=req.answer_text,
500             is_followup=is_followup,
501             is_retry=False,
502             duration_s=None,
503             wpm=None,
504             pause_cnt=None,
505             filler_cnt=filler_cnt,
506         )
507         db.add(new_answer)
508
509         new_report = None
510         if report_data:
511             new_report = Report(
512                 session_id=sid,
513                 dimensions_json=report_data["dimensions"],
514                 highlights_json=report_data["highlights"],
515                 concerns_json=report_data["concerns"],
516                 improvement_json=report_data["improvement"],
```

文本最终 token 条件提交 [独立逻辑块] / 续 / server/api/sessions.py

```
517         overall=report_data["overall"],
518         scoring_version=report_data.get("scoring_version"),
519     )
520     db.add(new_report)
521
522     db.commit()
523     if new_report:
524         db.refresh(new_report)
```

语音上传边界 [独立逻辑块]

server/api/sessions.py / 行 566-588

```
566     if not settings.asr_enabled:
567         raise HTTPException(
568             status_code=503,
569             detail={"code": "ASR_UNAVAILABLE", "message": "ASR 服务未启用"},
570         )
571
572     max_bytes = settings.asr_max_upload_mb * 1024 * 1024
573     # 最多读取 上限+1 字节: 足以判定超限, 又不把超大文件整体读入内存
574     audio_bytes = await audio.read(max_bytes + 1)
575     if len(audio_bytes) > max_bytes:
576         raise HTTPException(
577             status_code=413,
578             detail={"code": "AUDIO_TOO_LARGE", "message": "音频文件超过大小限制"},
579         )
580     try:
581         audio_service.validate_upload(audio_bytes)
582     except AudioInvalidError as exc:
583         raise HTTPException(
584             status_code=422,
585             detail={"code": "AUDIO_INVALID", "message": str(exc)},
586         )
587
588     temp_dir = tempfile.mkdtemp(prefix="asr_", dir=settings.asr_temp_dir or None)
```

语音转写进入同一推进链 [独立逻辑块]

server/api/sessions.py / 行 698-717

```
698     await audio_service.prepare_wav(webm_path, wav_path)
699     answer_text = await asr_service.transcribe_wav(wav_path)
700     if not answer_text.strip():
701         raise ASRUnavailableError("转写结果为空, 疑似无效音频")
702
703     filler_cnt = count_filler_words(answer_text)
704     wpm = _compute_wpm(answer_text, duration_s)
705     current_seq = pending_q["seq"]
706     is_followup = pending_q.get("is_followup", False)
707     current_q_text = pending_q["text"]
708
709     orig_q = (
710         questions[current_seq - 1]
711         if 1 <= current_seq <= len(questions)
712         else None
713     )
714     followup_hint = orig_q.followup_hint if orig_q else None
715     has_followup_answered = any(
716         a.q_seq == current_seq and a.is_followup for a in current_answers
717     )
```

语音最终 token 条件提交 [独立逻辑块]

server/api/sessions.py / 行 910-958

```
910         with SessionLocal() as db:
911             res = db.execute(
912                 update(Session)
913                 .where(Session.id == sid, Session.lease_token == my_token)
914                 .values(
915                     status=next_status,
916                     pending_question_json=next_q,
917                     lease_token=None,
918                     lease_expires_at=None,
919                     input_mode=func.coalesce(Session.input_mode, "voice"),
920                 )
921             )
922             if res.rowcount == 0:
923                 db.rollback()
924                 raise HTTPException(
925                     status_code=409,
926                     detail={"code": "ANSWER_IN_PROGRESS", "message": "答题租约已被后续请求接管"},
927                 )
928
929             new_answer = Answer(
930                 session_id=sid,
931                 q_seq=current_seq,
932                 question_text=current_q_text,
933                 answer_text=answer_text,
934                 is_followup=is_followup,
935                 is_retry=False,
936                 duration_s=duration_s,
937                 wpm=wpm,
938                 pause_cnt=pause_cnt,
939                 filler_cnt=filler_cnt,
940             )
941             db.add(new_answer)
942
943             new_report = None
944             if report_data:
945                 new_report = Report(
946                     session_id=sid,
947                     dimensions_json=report_data["dimensions"],
948                     highlights_json=report_data["highlights"],
949                     concerns_json=report_data["concerns"],
950                     improvement_json=report_data["improvement"],
951                     overall=report_data["overall"],
952                     scoring_version=report_data.get("scoring_version"),
953                 )
954             db.add(new_report)
955
956             db.commit()
957             if new_report:
958                 db.refresh(new_report)
```

05 ASR 与可复现声学规则

证明音频先转码再转写，文本归一化和填充词计数有确定规则；数值规则不等于测量效度证明。

音频转码

server/services/audio.py / 行 77-89

```
77 def transcode_to_16k_wav(webm_path: str, wav_path: str) -> None:
78     """阶段③: 探测通过后的转码；失败 → AudioProcessError (stderr 仅记日志)。"""
79     result = _run([
80         settings.ffmpeg_path,
81         "-hide_banner", "-loglevel", "error", "-y",
82         "-i", webm_path,
83         "-ac", "1", "-ar", str(settings.asr_sample_rate),
84         "-c:a", "pcm_s16le", wav_path,
85     ])
86     if result.returncode != 0:
87         stderr = result.stderr.decode("utf-8", "replace").strip()
88         logger.warning("ffmpeg 转码失败 (AUDIO_PROCESS_FAILED): %s", stderr[:200])
89         raise AudioProcessError("音频转码失败")
```

转写归一化

server/services/asr.py / 行 34-42

```
34 def normalize_asr_text(text: str) -> str:
35     """折叠 FunASR 常见的汉字间空白，保留英文词、数字及非 CJK - CJK 间隔。
36
37     只处理 CJK (\u4e00-\u9fff) 字符之间的一个或多个空白（含空格/制表/换行/全角空格）；
38     不修改 validate_evidence，不在此函数处理文本答题路径。
39     """
40     if not text:
41         return text
42     return _CJK_INTER_SPACE.sub("", text)
```

填充词计数

server/services/asr.py / 行 45-48

```
45 def count_filler_words(text: str) -> int:
46     """统计填充词（嗯|那个|就是|然后|这个）频次"""
47     pattern = r"(嗯|那个|就是|然后|这个)"
48     return len(re.findall(pattern, text))
```

语速计算

server/api/sessions.py / 行 543-548

```
543 def _compute_wpm(answer_text: str, duration_s: float) -> float | None:
544     """语速=去空白字数/时长×60 (AGENTS § 6.3)；时长非法时置空，不虚构。"""
545     if duration_s <= 0:
546         return None
547     chars = len(" ".join(answer_text.split()))
548     return round(chars / duration_s * 60, 1)
```

声学阈值常量 [独立逻辑块]

server/services/scoring.py / 行 86-96

```
86 # 表达流畅度确定性量化规则 (T5-FIX; 无 LLM 参与)
87 WPM_NORMAL_LOW = 220.0 # 正常语速区间下限 (字/分)
88 WPM_NORMAL_HIGH = 280.0 # 正常语速区间上限 (字/分)
89 WPM_PENALTY_PER_UNIT = 0.2 # 每偏离 1 字/分 扣 0.2 分
90 WPM_PENALTY_CAP = 40.0
91 PAUSE_FREE_PER_MIN = 6.0 # 每分钟停顿 ≤6 次不扣分
92 PAUSE_PENALTY_PER_UNIT = 2.0 # 超出部分每次扣 2 分 (按每分钟归一化)
93 PAUSE_PENALTY_CAP = 30.0
94 FILLER_FREE_PER_MIN = 5.0 # 每分钟填充词 ≤5 个不扣分
95 FILLER_PENALTY_PER_UNIT = 3.0 # 超出部分每个扣 3 分 (按每分钟归一化)
96 FILLER_PENALTY_CAP = 30.0
```

声学流畅度

server/services/scoring.py / 行 99-152

```
99 def compute_acoustic_fluency(samples: list[dict]) -> dict:
100     """基于声学特征 (语速/停顿/填充词) 的确定性表达流畅度评分, 可独立单测。
101
102     samples: 每条语音回答一条记录 {duration_s, wpm, pause_cnt, filler_cnt};
103     只统计声学字段完整的记录, 停顿/填充词按时长归一化为每分钟频率,
104     避免长回答天然吃亏; 语速按时长加权平均。
105     声学数据不足 (无完整记录) 时 score=None, 绝不虚构评分;
106     evidence 恒为 None: 声学数字不得冒充原文引用, 无合法原文引用时按契约置空。
107     """
108     complete = [
109         s for s in (samples or [])
110         if s.get("duration_s") is not None and float(s["duration_s"]) > 0
111         and s.get("wpm") is not None
112         and s.get("pause_cnt") is not None
113         and s.get("filler_cnt") is not None
114     ]
115     if not complete:
116         return {
117             "score": None,
118             "evidence": None,
119             "reason": "声学数据不足 (缺少时长/语速/停顿/填充词的完整记录), 未评估表达流畅度",
120         }
121
122     total_duration = sum(Decimal(str(s["duration_s"])) for s in complete)
123     wpm = float(
124         sum(Decimal(str(s["wpm"])) * Decimal(str(s["duration_s"])) for s in complete)
125         / total_duration
126     )
127     pause_per_min = float(
128         sum(Decimal(str(s["pause_cnt"])) for s in complete) * 60 / total_duration
129     )
130     filler_per_min = float(
131         sum(Decimal(str(s["filler_cnt"])) for s in complete) * 60 / total_duration
132     )
133
134     wpm_dev = max(0.0, WPM_NORMAL_LOW - wpm, wpm - WPM_NORMAL_HIGH)
135     wpm_penalty = min(WPM_PENALTY_CAP, wpm_dev * WPM_PENALTY_PER_UNIT)
136     pause_penalty = min(
137         PAUSE_PENALTY_CAP, max(0.0, pause_per_min - PAUSE_FREE_PER_MIN) * PAUSE_PENALTY_PER_UNIT
138     )
139     filler_penalty = min(
140         FILLER_PENALTY_CAP, max(0.0, filler_per_min - FILLER_FREE_PER_MIN) * FILLER_PENALTY_PER_UNIT
141     )
142
143     score = max(0.0, 100.0 - wpm_penalty - pause_penalty - filler_penalty)
144     score = float(Decimal(str(score)).quantize(Decimal("0.1"), rounding=ROUND_HALF_UP))
145
146     reason = (
```

声学流畅度 / 续 / server/services/scoring.py

```
147         f"语速{wpm:.0f}字/分（正常区间220-280，扣{wpm_penalty:.1f}分），"  
148         f"停顿{pause_per_min:.1f}次/分（≤6次/分不扣分，扣{pause_penalty:.1f}分），"  
149         f"填充词{filler_per_min:.1f}个/分（≤5个/分不扣分，扣{filler_penalty:.1f}分），"  
150         f"共{len(complete)}条有效语音回答"  
151     )  
152     return {"score": score, "evidence": None, "reason": reason}
```

06 评分证据与模型回退

证明两次独立评分、缺维空值、原文子串校验，以及三档模型回退与用量记录。

原文证据字面校验

server/services/scoring.py / 行 36-48

```
36 def validate_evidence(evidence: Optional[str], answers: list[str]) -> bool:
37     """严格校验引用原话证据:
38     1. 空白判断可以使用 strip, 但长度及子串匹配必须针对最终返回的原字符串;
39     2. 长度在 1..25 之间;
40     3. 必须为候选人某一条回答正文的原文字面连续子串（禁止跨回答拼接，禁止虚构）。
41     """
42     if not evidence or not isinstance(evidence, str):
43         return False
44     if not evidence.strip():
45         return False
46     if not (1 <= len(evidence) <= 25):
47         return False
48     return any(evidence in ans for ans in answers if ans)
```

维度合成

server/services/scoring.py / 行 51-66

```
51 def calculate_dimension_average(
52     d1: Optional[SingleDimensionScore],
53     d2: Optional[SingleDimensionScore],
54 ) -> tuple[Optional[float], Optional[str], str]:
55     """双评均值计算与字段继承:
56     1. 两次均有效时，转 Decimal 计算等权均值并 ROUND_HALF_UP 保留 1 位小数;
57     2. 固定继承第 1 次调用的 evidence 与 reason;
58     3. 任一次无效时，置 score=None, evidence=None, reason="未获得通过校验的双次评分"。
59     """
60     if d1 is None or d2 is None:
61         return None, None, "未获得通过校验的双次评分"
62
63     s1 = Decimal(str(d1.score))
64     s2 = Decimal(str(d2.score))
65     mean = ((s1 + s2) / Decimal("2")).quantize(Decimal("0.1"), rounding=ROUND_HALF_UP)
66     return float(mean), d1.evidence, d1.reason
```

有效维度总分

server/services/scoring.py / 行 69-83

```
69 def calculate_overall(dimensions: dict[str, dict]) -> Optional[float]:
70     """计算 overall: 对所有有效维度等权平均并四舍五入保留 1 位小数; 全缺失则返回 None"""
71     valid_scores = [
72         Decimal(str(d["score"]))
73         for d in dimensions.values()
74         if d.get("score") is not None
75     ]
76     if not valid_scores:
77         return None
78
79     total = sum(valid_scores)
80     mean = (total / Decimal(str(len(valid_scores)))).quantize(
81         Decimal("0.1"), rounding=ROUND_HALF_UP
82     )
83     return float(mean)
```

单次评分及校验

server/services/scoring.py / 行 155-203

```
155 async def call_scoring_llm(messages: list, answers: list[str]) -> SingleScoringResult:
```

单次评分及校验 / 续 / server/services/scoring.py

```
156 """单次调用评分模型并按维度校验证据合规性:
157 1. 某维度证据无效时发起一次性针对性重试（一次修正请求同时修正全部无效维度）;
158 2. 重试后依然不合规的维度置为 None, 逐维保留其余有效维度;
159 3. 真正底层 LLM 模型调用失败时向外抛出异常。
160 """
161 for attempt in range(2):
162     res = await chat_json("scoring", messages, SingleScoringResult)
163
164     # 逐维核对 evidence 是否合规
165     p_valid = bool(res.professional_match and validate_evidence(res.professional_match.evidence, answers))
166     l_valid = bool(res.logic_structure and validate_evidence(res.logic_structure.evidence, answers))
167     j_valid = bool(res.job_competence and validate_evidence(res.job_competence.evidence, answers))
168
169     invalid_dims = []
170     if res.professional_match and not p_valid:
171         invalid_dims.append("professional_match(专业匹配度)")
172     if res.logic_structure and not l_valid:
173         invalid_dims.append("logic_structure(逻辑结构)")
174     if res.job_competence and not j_valid:
175         invalid_dims.append("job_competence(岗位素养)")
176
177     # 若全部已评分维度的 evidence 均合规, 直接返回
178     if not invalid_dims:
179         return res
180
181     # 首次调用若存在不合规维度, 发起一次性针对性修正重试
182     if attempt == 0:
183         dims_str = ", ".join(invalid_dims)
184         messages.append({
185             "role": "user",
186             "content": (
187                 f"请注意: 以下维度的 evidence 未通过字面原话校验: {dims_str}。\\n"
188                 "evidence 必须是候选人某一单条回答正文中出现的字面连续子串（必须与候选人原话完全一致, 区分空格且不超过25字）, "
189                 "严禁修改原字词或跨句拼接。请重新输出完整的评分JSON（一次性修正上述维度的证据与评价）。"
190             ),
191         })
192     continue
193
194     # 重试后依然不合规的维度, 逐维置 None 保留其余有效维度
195     if res.professional_match and not p_valid:
196         res.professional_match = None
197     if res.logic_structure and not l_valid:
198         res.logic_structure = None
199     if res.job_competence and not j_valid:
200         res.job_competence = None
201     return res
202
203 return res
```

两次独立评分

server/services/scoring.py / 行 234-310

```
234 async def score_interview(
235     answers: list[str],
236     job_info: dict,
237     mode: str = "text",
238     acoustic_samples: Optional[list[dict]] = None,
239 ) -> dict:
240     """执行双评并仲裁汇总出最终报告数据:
241     1. 逐维保留有效结果; 某维缺失仅影响该维;
242     2. 两次均无有效证据生成全空报告;
243     3. 只有当真正模型链不可用（两次调用均抛出异常）时才报 503 ScoringUnavailableError。
244     acoustic_samples: 语音模式（mode="voice"）下各语音回答的声学记录
```

两次独立评分 / 续 / server/services/scoring.py

```

245     ({duration_s, wpm, pause_cnt, filler_cnt})), 表达流畅度据此确定性量化, 无第三次 LLM 调用。
246     """
247     messages_1 = build_scoring_prompt(answers, job_info)
248     messages_2 = build_scoring_prompt(answers, job_info)
249
250     # 尝试并行进行两次独立调用
251     results = await asyncio.gather(
252         call_scoring_llm(messages_1, answers),
253         call_scoring_llm(messages_2, answers),
254         return_exceptions=True,
255     )
256
257     # 只有底层模型基础设施调用均抛出异常, 才判定为基础设施不可用 (503)
258     if isinstance(results[0], Exception) and isinstance(results[1], Exception):
259         raise ScoringUnavailableError(f"所有评分模型均不可用: {results[0]}, {results[1]}")
260
261     res1 = results[0] if not isinstance(results[0], Exception) else None
262     res2 = results[1] if not isinstance(results[1], Exception) else None
263
264
265     # 计算各维度双评均值
266     prof_score, prof_ev, prof_reason = calculate_dimension_average(
267         res1.professional_match if res1 else None,
268         res2.professional_match if res2 else None,
269     )
270     logic_score, logic_ev, logic_reason = calculate_dimension_average(
271         res1.logic_structure if res1 else None,
272         res2.logic_structure if res2 else None,
273     )
274     comp_score, comp_ev, comp_reason = calculate_dimension_average(
275         res1.job_competence if res1 else None,
276         res2.job_competence if res2 else None,
277     )
278
279     # 表达流畅度: 文本模式保持 null; 语音模式由确定性声学规则量化 (数据不足时同样为 null, 不虚构)
280     if mode == "text":
281         fluency_dim = {
282             "score": None,
283             "evidence": None,
284             "reason": "文本模式, 未评估语音流畅度",
285         }
286     else:
287         fluency_dim = compute_acoustic_fluency(acoustic_samples or [])
288
289     dimensions = {
290         "professional_match": {"score": prof_score, "evidence": prof_ev, "reason": prof_reason},
291         "logic_structure": {"score": logic_score, "evidence": logic_ev, "reason": logic_reason},
292         "expression_fluency": fluency_dim,
293         "job_competence": {"score": comp_score, "evidence": comp_ev, "reason": comp_reason},
294     }
295
296     overall = calculate_overall(dimensions)
297
298     primary_res = res1 or res2
299     highlights = primary_res.highlights if primary_res else []
300     concerns = primary_res.concerns if primary_res else []
301     improvement = primary_res.improvement if primary_res else []
302
303     return {
304         "dimensions": dimensions,
305         "highlights": highlights,
306         "concerns": concerns,
307         "improvement": improvement,
308         "overall": overall,

```

两次独立评分 / 续 / server/services/scoring.py

```
309         "scoring_version": SCORING_VERSION,
310     }
```

模型回退链

server/services/llm.py / 行 39-45

```
39     def _chain(self, stage: str):
40         chain = [
41             ("flagship", getattr(self.settings, "flagship_model", None), getattr(self.settings, "flagship_base_url", None),
42              getattr(self.settings, "flagship_api_key", None)),
43             ("flash", getattr(self.settings, "flash_model", None), getattr(self.settings, "flash_base_url", None), getattr(
44                 self.settings, "flash_api_key", None)),
45             ("local", getattr(self.settings, "local_model", None), getattr(self.settings, "local_base_url", None), getattr(
46                 self.settings, "local_api_key", None)),
47         ]
48         return [entry for entry in chain if entry[1] and entry[2]]
```

调用、校验重试与用量

server/services/llm.py / 行 60-149

```
60     async def _attempts(self, stage, messages, response_model, deadline):
61         last = None
62         chain = self._chain(stage)
63         for index, (name, model, url, key) in enumerate(chain):
64             for retry in range(2):
65                 started = time.monotonic()
66                 record = {
67                     "stage": stage,
68                     "model": model,
69                     "success": False,
70                     "ttft_ms": None,
71                     "total_tokens": None,
72                     "error": None,
73                     "total_ms": 0.0,
74                 }
75                 if not url or not model:
76                     record["error"] = "missing_config"
77                     self._save_record(record)
78                     last = LLMError(f"missing config for {name}")
79                     break
80
81                 client = None
82                 try:
83                     remaining = deadline - time.monotonic()
84                     attempt_timeout = remaining
85                     if index < len(chain) - 1:
86                         # 仅为已配置的后续模型保留时间，空配置不能占用当前 API 的预算。
87                         shares_by_count = {
88                             1: (1.0,),
89                             2: (2 / 3, 1 / 3),
90                             3: (0.5, 1 / 3, 1 / 6),
91                         }
92                         shares = shares_by_count[len(chain)]
93                         total_budget = self._timeout(stage)
94                         fallback_reserve = total_budget * sum(shares[index + 1:])
95                         attempt_timeout = min(total_budget * shares[index], remaining - fallback_reserve)
96                         attempt_timeout = max(0.001, attempt_timeout)
97                         response_format = {"type": "json_object"}
98                         if name == "local":
99                             response_format = {
100                                 "type": "json_schema",
101                                 "json_schema": {
102                                     "name": response_model.__name__,
103                                     "strict": True,
```

调用、校验重试与用量 / 续 / server/services/llm.py

```

104         "schema": response_model.model_json_schema(),
105     },
106 }
107 request_options = {}
108 if stage == "answer_feedback" and urlparse(url).hostname == "api.deepseek.com":
109     request_options["extra_body"] = {"thinking": {"type": "disabled"}}
110     request_options["max_tokens"] = 512
111 client = AsyncOpenAI(base_url=url, api_key=key, max_retries=0)
112 resp = await asyncio.wait_for(client.chat.completions.create(
113     model=model,
114     messages=messages,
115     response_format=response_format,
116     timeout=attempt_timeout,
117     **request_options,
118 ), timeout=attempt_timeout)
119 # Non-streaming client: this is the first point at which response content is available.
120 record["tft_ms"] = round((time.monotonic() - started) * 1000, 1)
121 usage = getattr(resp, "usage", None)
122 record["total_tokens"] = getattr(usage, "total_tokens", None)
123 record["prompt_tokens"] = getattr(usage, "prompt_tokens", None)
124 record["completion_tokens"] = getattr(usage, "completion_tokens", None)
125 data = resp.choices[0].message.content
126 result = response_model.model_validate_json(data)
127 record["success"] = True
128 return result
129 except ValidationError as e:
130     last = e
131     record["error"] = "validation_error"
132     if retry == 0:
133         continue
134 except asyncio.CancelledError:
135     record["error"] = "stage_deadline_or_cancelled"
136     raise
137 except Exception as e:
138     last = e
139     record["error"] = type(e).__name__
140     break
141 finally:
142     record["total_ms"] = round((time.monotonic() - started) * 1000, 1)
143     self._save_record(record)
144     if client is not None and hasattr(client, "close"):
145         try:
146             await asyncio.wait_for(client.close(), timeout=0.25)
147         except (Exception, asyncio.CancelledError):
148             pass
149     raise LLMError("all llm attempts failed") from last

```

07 成长记录与曲线

证明历史按本人同岗查询，综合分按严格口径比较；同版本且输入模式已知时可比较共同的非声学维度。

已完成报告筛选

server/services/growth.py / 行 85-100

```
85 def iter_qualified(  
86     db: DbSession,  
87     user_id: int,  
88     job_id: int | None = None,  
89 ) -> list[tuple[Session, Report, Job]]:  
90     """completed 且有 report 的会话，按 started_at, session.id 升序。"""  
91     q = (  
92         db.query(Session, Report, Job)  
93         .join(Report, Report.session_id == Session.id)  
94         .join(Job, Job.id == Session.job_id)  
95         .filter(Session.user_id == user_id, Session.status == "completed")  
96     )  
97     if job_id is not None:  
98         q = q.filter(Session.job_id == job_id)  
99     rows = q.order_by(Session.started_at.asc(), Session.id.asc()).all()  
100    return list(rows)
```

综合分可比性

server/services/growth.py / 行 145-190

```
145 def _compare_overall(prev: Session, prev_r: Report, curr: Session, curr_r: Report) -> dict:  
146     reasons: list[str] = []  
147     prev_scores = _score_map(prev_r.dimensions_json)  
148     curr_scores = _score_map(curr_r.dimensions_json)  
149     prev_eff = _effective_dims(prev_scores)  
150     curr_eff = _effective_dims(curr_scores)  
151  
152     if prev_r.overall is None or curr_r.overall is None:  
153         reasons.append("OVERALL_MISSING")  
154  
155     if prev.input_mode is None or curr.input_mode is None:  
156         reasons.append("INPUT_MODE_UNKNOWN")  
157     elif prev.input_mode != curr.input_mode:  
158         reasons.append("INPUT_MODE_MISMATCH")  
159  
160     if prev_r.scoring_version is None or curr_r.scoring_version is None:  
161         reasons.append("SCORING_VERSION_UNKNOWN")  
162     elif prev_r.scoring_version != curr_r.scoring_version:  
163         reasons.append("SCORING_VERSION_MISMATCH")  
164  
165     if not prev_eff and not curr_eff:  
166         reasons.append("EMPTY_DIMENSION_SET")  
167     elif prev_eff != curr_eff:  
168         reasons.append("DIMENSION_SET_MISMATCH")  
169  
170     comparable = len(reasons) == 0  
171     delta = None  
172     if comparable:  
173         delta = _round1(float(curr_r.overall) - float(prev_r.overall))  
174  
175     message = None  
176     if reasons:  
177         # 优先使用首个原因的标准文案；维度集合不同用规格示例语气  
178         if "DIMENSION_SET_MISMATCH" in reasons:  
179             message = REASON_MESSAGES["DIMENSION_SET_MISMATCH"]  
180         else:  
181             message = REASON_MESSAGES.get(reasons[0])
```

综合分可比性 / 续 / server/services/growth.py

```
182
183     return {
184         "comparable": comparable,
185         "reasons": reasons,
186         "message": message,
187         "previous": {"session_id": prev.id, "overall": prev_r.overall},
188         "current": {"session_id": curr.id, "overall": curr_r.overall},
189         "delta": delta,
190     }
```

共同有效维度

server/services/growth.py / 行 193-208

```
193 def _dimension_changes(prev_r: Report, curr_r: Report) -> dict[str, dict | None]:
194     prev_scores = _score_map(prev_r.dimensions_json)
195     curr_scores = _score_map(curr_r.dimensions_json)
196     out: dict[str, dict | None] = {}
197     for key in DIMENSION_KEYS:
198         p = prev_scores[key]
199         c = curr_scores[key]
200         if p is None or c is None:
201             out[key] = None
202         else:
203             out[key] = {
204                 "previous": float(p),
205                 "current": float(c),
206                 "delta": _round1(float(c) - float(p)),
207             }
208     return out
```

成长趋势与跨模式维度门禁

server/services/growth.py / 行 211-281

```
211 def build_trend(db: DbSession, user_id: int, job_id: int) -> dict:
212     require_student(db, user_id)
213     job = require_job(db, job_id)
214     rows = iter_qualified(db, user_id, job_id)
215
216     points = []
217     for sess, report, _job in rows:
218         points.append(
219             {
220                 "session_id": sess.id,
221                 "report_id": report.id,
222                 "started_at": _format_started_at(sess.started_at),
223                 "overall": report.overall,
224                 "dimensions": _score_map(report.dimensions_json),
225             }
226         )
227
228     modes = [sess.input_mode for sess, _r, _j in rows]
229     top_mode = _aggregate_input_mode(modes)
230
231     if len(rows) == 0:
232         comparison = {
233             "comparable": False,
234             "reasons": ["NO_RECORDS"],
235             "message": REASON_MESSAGES["NO_RECORDS"],
236             "previous": None,
237             "current": None,
238             "delta": None,
239         }
240     dim_changes = {k: None for k in DIMENSION_KEYS}
241     elif len(rows) == 1:
```

成长趋势与跨模式维度门禁 / 续 / server/services/growth.py

```
242         comparison = {
243             "comparable": False,
244             "reasons": ["SINGLE_RECORD"],
245             "message": REASON_MESSAGES["SINGLE_RECORD"],
246             "previous": None,
247             "current": None,
248             "delta": None,
249         }
250         dim_changes = {k: None for k in DIMENSION_KEYS}
251     else:
252         prev_s, prev_r, _ = rows[-2]
253         curr_s, curr_r, _ = rows[-1]
254         comparison = _compare_overall(prev_s, prev_r, curr_s, curr_r)
255         known_input_modes = (
256             prev_s.input_mode is not None
257             and curr_s.input_mode is not None
258         )
259         same_scoring_version = (
260             prev_r.scoring_version is not None
261             and prev_r.scoring_version == curr_r.scoring_version
262         )
263         dim_changes = (
264             _dimension_changes(prev_r, curr_r)
265             if known_input_modes and same_scoring_version
266             else {key: None for key in DIMENSION_KEYS}
267         )
268         # 声学流畅度不跨文本与语音口径比较；其余同版本有效维度仍可逐项观察。
269         if prev_s.input_mode != curr_s.input_mode:
270             dim_changes["expression_fluency"] = None
271
272     return {
273         "user_id": user_id,
274         "job_id": job_id,
275         "job_title": job.title,
276         "input_mode": top_mode,
277         "sessions_count": len(rows),
278         "points": points,
279         "overall_comparison": comparison,
280         "dimension_changes": dim_changes,
281     }
```

综合分分组键 [独立逻辑块]

web/lib/growth-data.ts / 行 609-615

```
609 function overallGroupKey(point: TimelinePoint): string {
610     const validDimensions = REQUIRED_DIMENSIONS.filter((dim) => point.dimensions[dim] !== null);
611     if (point.inputMode === null || !point.scoringVersion?.trim() || validDimensions.length === 0) {
612         return `session:${point.sessionId}`;
613     }
614     return JSON.stringify([point.inputMode, point.scoringVersion, validDimensions]);
615 }
```

缺失点与分口径曲线 [独立逻辑块]

web/lib/growth-data.ts / 行 617-743

```
617 /**
618  * 折线图: null 保持为 null (不补 0); connectNulls 恒为 false。
619  * 综合分仅在已知输入模式、评分版本和有效维度集合相同的记录间连线。
620  * 非声学有效维度可在同一已知评分版本内跨输入模式连线。
621  * 综合分未知口径或空有效维度集合保留独立点; 各维度 null 不补零、不跨 null 连线。
622  * 单 cohort 且有效维度集合一致时保持一条综合分 + 四维折线。
623  */
624 export function transformTrendToLineOptions(
625     trend: GrowthTrendData,
```

缺点点与分口径曲线 [独立逻辑块] / 续 / web/lib/growth-data.ts

```

626   historyRecords: GrowthHistoryRecord[] = [],
627   theme: "day" | "night" = "night",
628 ): Record<string, unknown> {
629   const labelColor = theme === "day" ? "#626b7d" : "#9fb4d4";
630   const gridColor = theme === "day" ? "rgba(105, 84, 190, 0.18)" : "rgba(47, 111, 237, 0.2)";
631   const connectNulls = false;
632   const timeline = buildTimeline(trend, historyRecords);
633   const categories = timeline.map((p) => `${p.sessionId}`);
634   const cohortKeys = uniqueCohortKeys(timeline);
635   const multiCohort = cohortKeys.length > 1;
636   const overallKeys = [...new Set(timeline.map(overallGroupKey))];
637   const overallBaseName = (point: TimelinePoint) =>
638     `${multiCohort ? `${cohortDisplayLabel(point.inputMode, point.scoringVersion)} ` : ""}综合分`;
639   const overallSeries = overallKeys.map((key) => {
640     const first = timeline.find((point) => overallGroupKey(point) === key)!;
641     const baseName = overallBaseName(first);
642     const duplicateName = overallKeys.some((otherKey) =>
643       otherKey !== key && overallBaseName(timeline.find((point) => overallGroupKey(point) === otherKey)!) === baseName,
644     );
645     const validDimensions = REQUIRED_DIMENSIONS.filter((dim) => first.dimensions[dim] !== null);
646     const detail = key.startsWith("session:")
647       ? `${first.sessionId}`
648       : validDimensions.map((dim) => DIMENSION_LABEL_MAP[dim]).join(", ");
649     return {
650       name: duplicateName ? `${baseName} (${detail})` : baseName,
651       type: "line",
652       connectNulls,
653       data: timeline.map((point) => overallGroupKey(point) === key ? point.overall : null),
654     };
655   });
656
657   const baseChart = {
658     tooltip: {
659       trigger: "axis",
660       backgroundColor: theme === "day" ? "rgba(255, 255, 255, 0.96)" : "rgba(15, 23, 42, 0.92)",
661       borderColor: theme === "day" ? "rgba(112, 83, 239, 0.28)" : "rgba(47, 111, 237, 0.28)",
662       textStyle: { color: theme === "day" ? "#24243a" : "#e7ecfb" },
663     },
664     yAxis: {
665       type: "value",
666       min: 0,
667       max: 100,
668       axisLabel: { color: labelColor },
669       splitLine: { lineStyle: { color: gridColor } },
670     },
671   };
672
673   if (!multiCohort) {
674     const dimensionSeries = REQUIRED_DIMENSIONS.map((key) => ({
675       name: DIMENSION_LABEL_MAP[key],
676       type: "line",
677       connectNulls,
678       data: timeline.map((p) => p.dimensions[key]),
679     }));
680     return {
681       ...baseChart,
682       legend: {
683         data: [...overallSeries.map((series) => series.name), ...REQUIRED_DIMENSIONS.map((k) => DIMENSION_LABEL_MAP[k])],
684         textStyle: { color: labelColor },
685       },
686       xAxis: {
687         type: "category",
688         data: categories,
689         axisLabel: { color: labelColor },

```

缺点点与分口径曲线 [独立逻辑块] / 续 / web/lib/growth-data.ts

```

690     },
691     series: [...overallSeries, ...dimensionSeries],
692     _meta: { connectNulls: false, mixed: false, multiCohort: false, cohortCount: cohortKeys.length },
693   };
694 }
695
696 const series: Array<Record<string, unknown>> = [...overallSeries];
697 const legendData: string[] = overallSeries.map((item) => item.name);
698
699 const dimensionGroupKey = (point: TimelinePoint) =>
700   point.inputMode !== null && point.scoringVersion !== null
701     ? `version:${point.scoringVersion}`
702     : `cohort:${cohortIdentityKey(point.inputMode, point.scoringVersion)}`;
703 const dimensionKeys = [...new Set(timeline.map(dimensionGroupKey))];
704 for (const key of dimensionKeys) {
705   const members = timeline.filter((point) => dimensionGroupKey(point) === key);
706   const first = members[0];
707   const modes = new Set(members.map((point) => point.inputMode));
708   const label = modes.size > 1 && first.scoringVersion !== null
709     ? first.scoringVersion
710     : cohortDisplayLabel(first.inputMode, first.scoringVersion);
711   for (const dim of REQUIRED_DIMENSIONS) {
712     if (modes.size > 1 && dim === "expression_fluency") continue;
713     const dimName = `${label} ${DIMENSION_LABEL_MAP[dim]}`;
714     legendData.push(dimName);
715     series.push({
716       name: dimName,
717       type: "line",
718       connectNulls,
719       data: timeline.map((point) =>
720         dimensionGroupKey(point) === key ? point.dimensions[dim] : null,
721       ),
722     });
723   }
724 }
725
726 return {
727   ...baseChart,
728   legend: { data: legendData, textStyle: { color: labelColor } },
729   xAxis: {
730     type: "category",
731     data: categories,
732     axisLabel: { color: labelColor },
733   },
734   series,
735   _meta: {
736     connectNulls: false,
737     mixed: true,
738     multiCohort: true,
739     cohortCount: cohortKeys.length,
740   },
741 };
742 }
743

```

08 企业同源初筛

证明候选人来自真实会话报告，每人同岗位取最近合格记录；权重含学历学校且可配置。

六因素权重与旧配置默认

server/services/recruiter.py / 行 36-82

```
36 def parse_job_weights(dims_json: Any) -> dict[str, float]:
37     """Validate six-factor weights; legacy four-factor jobs get 60/20/20 defaults."""
38     if not isinstance(dims_json, dict):
39         raise JobWeightsInvalid("dims_json 必须为对象")
40     weights = dims_json.get("weights")
41     if not isinstance(weights, dict):
42         raise JobWeightsInvalid("dims_json.weights 必须为对象")
43     weight_keys = set(weights.keys())
44     if weight_keys == set(DIM_KEYS):
45         legacy_values: dict[str, Decimal] = {}
46         legacy_total = Decimal("0")
47         for key in DIM_KEYS:
48             raw = weights[key]
49             if isinstance(raw, bool) or not isinstance(raw, (int, float)):
50                 raise JobWeightsInvalid(f"权重 {key} 必须为 number")
51             value = Decimal(str(raw))
52             if not value.is_finite() or value < 0:
53                 raise JobWeightsInvalid(f"权重 {key} 必须为有限的非负数")
54             legacy_values[key] = value
55             legacy_total += value
56         if abs(legacy_total - Decimal("1")) > Decimal("0.000001"):
57             raise JobWeightsInvalid("旧版四项权重之和必须为 1.0")
58         weights = {
59             **{key: float(value * Decimal("0.6")) for key, value in legacy_values.items()},
60             "education_level": 0.2,
61             "school_tier": 0.2,
62         }
63     elif weight_keys != set(WEIGHT_KEYS):
64         raise JobWeightsInvalid("weights 必须包含四项面试维度、学历和学校档次")
65     values: dict[str, float] = {}
66     total = Decimal("0")
67     for key in WEIGHT_KEYS:
68         raw = weights[key]
69         if isinstance(raw, bool) or not isinstance(raw, (int, float)):
70             raise JobWeightsInvalid(f"权重 {key} 必须为 number")
71         val = Decimal(str(raw))
72         if not val.is_finite() or val < 0:
73             raise JobWeightsInvalid(f"权重 {key} 必须为有限的非负数")
74         values[key] = float(val)
75         total += val
76     if abs(total - Decimal("1")) > Decimal("0.000001"):
77         raise JobWeightsInvalid("权重之和必须为 1.0")
78     labels = dims_json.get("labels")
79     if labels is not None:
80         if not isinstance(labels, list) or len(labels) != 4:
81             raise JobWeightsInvalid("labels 若存在长度须为 4")
82     return values
```

有效项加权

server/services/recruiter.py / 行 105-133

```
105 def compute_weighted_score(
106     dimensions: dict | None,
107     weights: dict[str, float],
108     education_level: str | None = None,
109     school_tier: str | None = None,
110 ) -> float | None:
```

有效项加权 / 续 / server/services/recruiter.py

```
111     parts: list[tuple[Decimal, Decimal]] = []
112     for key in DIM_KEYS:
113         cell = dimensions.get(key) if isinstance(dimensions, dict) else None
114         if not isinstance(cell, dict):
115             continue
116         score = cell.get("score")
117         if not is_valid_dimension_score(score) or weights.get(key, 0.0) <= 0:
118             continue
119         parts.append((Decimal(str(score)), Decimal(str(weights[key]))))
120     profile_scores = {
121         "education_level": EDUCATION_SCORES.get(education_level or ""),
122         "school_tier": SCHOOL_TIER_SCORES.get(school_tier or ""),
123     }
124     for key, score in profile_scores.items():
125         if score is not None and weights.get(key, 0.0) > 0:
126             parts.append((Decimal(str(score)), Decimal(str(weights[key]))))
127     if not parts:
128         return None
129     weight_sum = sum(w for _, w in parts)
130     if weight_sum == 0:
131         return None
132     total = sum(s * (w / weight_sum) for s, w in parts)
133     return float(total.quantize(Decimal("0.1"), rounding=ROUND_HALF_UP))
```

同源报告查询

server/services/recruiter.py / 行 168-184

```
168 def _eligible_reports_for_cohort(
169     db: DbSession, job_id: int, input_mode: str, scoring_version: str
170 ) -> list[tuple[Session, Report, User]]:
171     return list(
172         db.query(Session, Report, User)
173         .join(Report, Report.session_id == Session.id)
174         .join(User, User.id == Session.user_id)
175         .filter(
176             Session.job_id == job_id,
177             Session.status == "completed",
178             User.role == "student",
179             Session.input_mode == input_mode,
180             Report.scoring_version == scoring_version,
181         )
182         .order_by(Session.started_at.desc(), Session.id.desc())
183         .all()
184     )
```

每人最近一份

server/services/recruiter.py / 行 187-198

```
187 def pick_latest_eligible_per_user(
188     rows: list[tuple[Session, Report, User]],
189 ) -> list[tuple[Session, Report, User]]:
190     """按 started_at DESC, id DESC 扫描, 每用户取首条 valid_dim_count>=3。"""
191     chosen: dict[int, tuple[Session, Report, User]] = {}
192     for sess, report, user in rows:
193         if user.id in chosen:
194             continue
195         if valid_dim_count(report.dimensions_json) < MIN_VALID_DIMS:
196             continue
197         chosen[user.id] = (sess, report, user)
198     return list(chosen.values())
```

候选人与溯源标识

server/services/recruiter.py / 行 201-253

```
201 def build_candidate_item(  
202     sess: Session,  
203     report: Report,  
204     user: User,  
205     weights: dict[str, float],  
206 ) -> dict:  
207     raw = report.dimensions_json if isinstance(report.dimensions_json, dict) else {}  
208     dimensions: dict[str, dict] = {}  
209     for key in DIM_KEYS:  
210         cell = raw.get(key)  
211         if not isinstance(cell, dict):  
212             dimensions[key] = {  
213                 "score": None,  
214                 "evidence": None,  
215                 "reason": "未评估",  
216             }  
217             continue  
218         score = cell.get("score")  
219         if not is_valid_dimension_score(score):  
220             score = None  
221         evidence = cell.get("evidence")  
222         if evidence is not None and not isinstance(evidence, str):  
223             evidence = None  
224         reason = cell.get("reason")  
225         if not isinstance(reason, str) or not reason:  
226             reason = "未评估" if score is None else "—"   
227         dimensions[key] = {  
228             "score": float(score) if score is not None else None,  
229             "evidence": evidence,  
230             "reason": reason,  
231         }  
232     return {  
233         "user_id": user.id,  
234         "name_masked": user.name_masked,  
235         "major": user.major,  
236         "grade": user.grade,  
237         "student_no": user.student_no,  
238         "education_level": user.education_level,  
239         "school_tier": user.school_tier,  
240         "session_id": sess.id,  
241         "report_id": report.id,  
242         "job_id": sess.job_id,  
243         "input_mode": sess.input_mode,  
244         "scoring_version": report.scoring_version,  
245         "trained_at": _format_started_at(sess.started_at),  
246         "overall": report.overall,  
247         "dimensions": dimensions,  
248         "valid_dim_count": valid_dim_count(raw),  
249         "weighted_score": compute_weighted_score(  
250             raw, weights, user.education_level, user.school_tier  
251         ),  
252         "report_path": f"/reports/{sess.id}",  
253     }
```

候选排序

server/services/recruiter.py / 行 272-329

```
272 def build_candidates_response(  
273     db: DbSession,  
274     *,  
275     job_id: int,  
276     input_mode: str | None,  
277     scoring_version: str | None,
```

候选排序 / 续 / server/services/recruiter.py

```
278 ) -> dict:
279     job = db.query(Job).filter(Job.id == job_id).first()
280     if job is None:
281         raise LookupError("JOB_NOT_FOUND")
282
283     available = list_available_cohorts(db, job_id)
284     weights_error = None
285     weights_payload = None
286     raw_weights: dict[str, float] | None
287     try:
288         raw_weights = parse_job_weights(job.dims_json)
289         weights_payload = {"scheme": "job_dims_renorm", **raw_weights}
290     except JobWeightsInvalid:
291         weights_error = "JOB_WEIGHTS_INVALID"
292         raw_weights = None
293
294     # 发现模式: 仅 job_id
295     if input_mode is None and scoring_version is None:
296         return {
297             "job_id": job.id,
298             "job_title": job.title,
299             "cohort": None,
300             "available_cohorts": available,
301             "weights": weights_payload,
302             "weights_error": weights_error,
303             "eligibility": ELIGIBILITY,
304             "candidates": [],
305         }
306
307     # 完整模式: 权重非法 → 由 API 转为 503
308     if raw_weights is None:
309         raise JobWeightsInvalid(weights_error or "JOB_WEIGHTS_INVALID")
310
311     rows = _eligible_reports_for_cohort(db, job_id, input_mode, scoring_version)
312     picked = pick_latest_eligible_per_user(rows)
313     candidates = _sort_candidates(
314         [
315             build_candidate_item(sess, report, user, raw_weights)
316             for sess, report, user in picked
317         ]
318     )
319
320     return {
321         "job_id": job.id,
322         "job_title": job.title,
323         "cohort": {"input_mode": input_mode, "scoring_version": scoring_version},
324         "available_cohorts": available,
325         "weights": weights_payload,
326         "weights_error": None,
327         "eligibility": ELIGIBILITY,
328         "candidates": candidates,
329     }
```

09 新生咨询与学习

证明咨询连接现有岗位；学习菜单让新生先识别题型，再写自己的答案，STAR 只用于经历题。

岗位路径咨询

server/services/counsel.py / 行 45-140

```
45  async def build_counsel_response(db: Session, request: CounselRequest) -> CounselResponse:
46      static = json.loads(STATIC_PATH.read_text(encoding="utf-8"))
47      definitions = static["jobs"]
48      titles = [item["seed_key"] for item in definitions]
49      jobs = db.query(Job).filter(Job.title.in_(titles)).order_by(Job.id).all()
50      by_title = {job.title: job for job in jobs}
51      if len(jobs) != 2 or any(title not in by_title for title in titles):
52          raise LookupError("SEED_JOBS_UNAVAILABLE")
53
54      interest_text = " ".join(request.interests).lower()
55      fallback_id = next(
56          (
57              by_title[item["seed_key"]].id
58              for item in definitions
59              if any(k.lower() in interest_text for k in item["interest_keywords"])
60          ),
61          jobs[0].id,
62      )
63
64      degraded = False
65      result: CounselLlmResponse | None = None
66      try:
67          raw = await chat_json(
68              "counsel",
69              [
70                  {
71                      "role": "system",
72                      "content": "只在给定两个岗位内组织新生建议，禁止给分或发明岗位。",
73                  },
74                  {
75                      "role": "user",
76                      "content": json.dumps(
77                          {
78                              "major": request.major,
79                              "grade": request.grade,
80                              "interests": request.interests,
81                              "jobs": [{"job_id": j.id, "title": j.title} for j in jobs],
82                          },
83                          ensure_ascii=False,
84                      ),
85                  },
86              ],
87              CounselLlmResponse,
88          )
89          if isinstance(raw, CounselLlmResponse):
90              result = raw
91          else:
92              result = CounselLlmResponse.model_validate(raw)
93      except Exception:
94          degraded = True
95          result = None
96
97      valid_ids = {job.id for job in jobs}
98      recommended = (
99          result.recommended_job_id
100          if result and result.recommended_job_id in valid_ids
101          else fallback_id
```

岗位路径咨询 / 续 / server/services/counsel.py

```
102     )
103     if result and result.recommended_job_id not in valid_ids:
104         degraded = True
105
106     job_map: list[CounselJobMapItem] = []
107     gaps: list[CounselGapItem] = []
108     for definition in definitions:
109         job = by_title[definition["seed_key"]]
110         job_map.append(
111             CounselJobMapItem(
112                 job_id=job.id,
113                 family=job.family,
114                 title=job.title,
115                 chain_role=definition["chain_role"],
116                 core_skills=definition["core_skills"],
117                 fit_summary=_resolve_fit(result, job.id),
118             )
119         )
120     for gap in definition["gap_templates"]:
121         gaps.append(
122             CounselGapItem(
123                 job_id=job.id,
124                 skill=gap["skill"],
125                 current_hint="结合当前课程与项目经历自查",
126                 target_hint=gap["target_hint"],
127                 suggested_action="完成基础学习后进入面试仓训练并复盘",
128             )
129         )
130
131     learning_path = _static_learning_path(static, request.grade)
132     return CounselResponse(
133         mode="新生",
134         degraded=degraded,
135         job_map=sorted(job_map, key=lambda item: item.job_id),
136         gaps=gaps,
137         learning_path=learning_path,
138         recommended_job_id=recommended,
139         train_hint=CounselTrainHint(job_id=recommended, mode="新生"),
140     )
```

题型识别练习 [独立逻辑块]

web/lib/learning.ts / 行 20-39

```
20 export const questionExercises = [
21     {
22         id: "project", kind: "experience" as const,
23         question: "请讲一次课程项目遇到困难，你是怎样处理的？",
24         options: [
25             { label: "说明困难、本人采取的行动，以及实际结果", correct: true, explanation: "题目要听一段具体经历。先交代问题，再重点讲自己的做法和结果；可以用 STAR 组织。" },
26             { label: "介绍课程学过的所有知识点", correct: false, explanation: "只罗列知识点，没有回应“遇到什么困难、怎样处理”，面试官仍听不出你的行动。" },
27             { label: "只说团队很努力，最后顺利完成", correct: false, explanation: "这句话缺少困难细节和个人贡献。请说清你负责的步骤与真实结果。" },
28         ],
29     },
30     {
31         id: "knowledge", kind: "knowledge" as const,
32         question: "请比较台架测试与实地测试各自适合解决什么问题。",
33         options: [
34             { label: "先给比较结论，再说两者各自的适用场景", correct: true, explanation: "这是比较题。先分清两种测试各自能验证什么，再举例；无需硬套一段个人经历。" },
35             { label: "先讲一段自己参加社团的完整 STAR 故事", correct: false, explanation: "这道题问的是概念与适用场景。STAR 用于经历类题，这里先直接比较两者。" },
36         ],
37     },
38 ]
```

题型识别练习 [独立逻辑块] / 续 / web/lib/learning.ts

```
36     { label: "只回答两者都很重要", correct: false, explanation: "两者都重要并没有回答区别。至少说明各自侧重和一个典型场景。" },
37   ],
38 },
39 ];
```

学习菜单与 STAR 边界 [独立逻辑块]

web/lib/learning.ts / 行 41-50

```
41 export const learningLessons: {
42   id: LearningTopic; title: string; summary: string; mistake: string; method: string[]; example: string; practice: string
43 }[] = [
44   { id: "hear", title: "先听懂面试官在问什么", summary: "找动作词、对象和范围,再开口。", mistake: "听见熟悉词就讲经历,却漏了“为什么”“如何比较”等关键要求。", method: ["抓动作词:介绍、比较、说明原因、举例。", "把两部分问题分开回答;必要时先用一句话确认。", "先给直接回答,再补一条依据或例子。"], example: "问“遇到什么困难、怎样处理”时,先交代困难,再说自己的处理步骤。", practice: "找一道题,写下“要做什么、围绕什么、答几部分”。" },
45   { id: "star", title: "STAR: 把真实经历讲清楚", summary: "经历题讲情境、任务、行动、结果。", mistake: "每题都套 STAR,铺垫过长,或把团队工作全说自己做的。", method: ["S 情境和 T 任务尽量简短。", "A 行动是重点:讲清楚自己做了什么。", "R 结果只写真实发生的事;没有数字也能说明反馈与复盘。"], example: "课程展示前资料格式不一;我负责汇总,建立清单核对分工;按时交稿后发现需要更早约定格式。", practice: "选一段真实经历,分别写四句。" },
46   { id: "intro", title: "一分钟自我介绍", summary: "专业、相关经历、目标,给面试官留下可追问的线索。", mistake: "从头背简历,只说“认真负责”,没有具体例子。", method: ["说清自己在学什么。", "挑一件与岗位相关、确实做过的事。", "说明这次训练想了解或争取什么。"], example: "“我学习车辆工程,课程项目中负责整理雨天视频并统计漏检情况。我希望了解测试岗位如何验证边界场景。”", practice: "录一段约一分钟的介绍,检查听者能否复述你的专业、例子与目标。" },
47   { id: "followup", title: "接住追问与请求澄清", summary: "先直接回答,再补具体事实。", mistake: "把追问当否定,一遍遍重复原答案;没听清时猜着答。", method: ["不确定题意时,礼貌确认关键条件。", "追问本人贡献时,区分自己和团队做的事。", "一句话答核心,再补一个可以核实的细节。"], example: "“您想了解的是我负责的部分,对吗?我主要核对数据来源,并整理成可复查的清单。”", practice: "请同伴连续追问一段经历两次,每次先用一句话答重点。" },
48   { id: "unknown", title: "不会的问题也能诚实回答", summary: "说明边界、相关基础和补学方法。", mistake: "不懂也硬答,编造项目、操作记录或无法核对的数字。", method: ["明确目前没实际接触过什么。", "说出真正相关的课程或相近练习。", "说明会如何查规范、验证和求助。"], example: "“这个工具我还没实际用过。我做过相近的数据整理,会先看操作规范,再用练习数据熟悉流程。”", practice: "找一个陌生术语,用三句话说“不熟的部分、已有基础、下一步”。" },
49   { id: "review", title: "读懂反馈,再练一次", summary: "核对原话和分析,挑一个具体动作改进。", mistake: "只盯总分,把一次模型评分当成对能力的最终结论。", method: ["先核对引用是否真出自你的回答。", "再看分析是否对应这段话;有疑问时不要盲从。", "选一条可操作的建议,重答同类题并回看变化。"], example: "若回答只说“我们完成了项目”,可补自己负责的步骤、材料和真实结果。", practice: "从报告选一条建议,指出支撑它的回答原话,再重答一次。" },
50 ];
```

识别-诊断-搭建-自写练习 [独立逻辑块]

web/app/learn/learn-content.tsx / 行 118-170

```
118 <section className={styles.workshop} aria-labelledby="workshop-title">
119   <p className={styles.eyebrow}>互动工坊 · 参考四步递进练习</p>
120   <h2 id="workshop-title">识别 → 诊断 → 搭建 → 写自己的答案</h2>
121   <p className={styles.lead}>练习会立即解释选择理由。这里的检查只帮助整理答案,不代表面试评分。</p>
122
123   <div className={styles.workshopStep}>
124     <div className={styles.stepTitle}><span>01</span><h3>识别: 面试官真正想听什么? </h3></div>
125     {questionExercises.map((q) => <fieldset key={q.id} className={styles.exercise}>
126       <legend>{q.question}</legend>
127       {q.options.map((option, idx) => <label key={option.label} className={styles.choice}>
128         <input type="radio" name={q.id} checked={answers[q.id] === idx} onChange={() => setAnswers((prev) => ({ ...prev, [q.id]: idx })))} />
129         <span>{option.label}</span>
130       </label>)}
131       {answers[q.id] !== undefined && <p className={styles.explanation} role="status"><strong>{q.options[answers[q.id]].correct ? "答对了: " : "再想想: "}</strong>{q.options[answers[q.id]].explanation}</p>
132     </fieldset>)}
133   </div>
134
135   <div className={styles.workshopStep}>
136     <div className={styles.stepTitle}><span>02</span><h3>诊断: 这段回答最大的问题是什么? </h3></div>
137     <p className={styles.question}>{diagnose.question}</p><blockquote>{diagnose.answer}</blockquote>
```

识别-诊断-搭建-自写练习 [独立逻辑块] / 续 / web/app/learn/learn-content.tsx

```
138     <fieldset className={styles.exercise}><legend>选出最需要先改的地方</legend>
139     {diagnose.choices.map((choice, idx) => <label key={choice.label} className={styles.choice}>
140       <input type="radio" name="diagnose" checked={diagnosis === idx} onChange={() => setDiagnosis(idx)} /><span>
141         {choice.label}</span>
142       </label>)}
143     {diagnosis !== null && <p className={styles.explanation} role="status"><strong>[diagnose.choices[diagnosis].correct ? "抓到重点了: " : "看看这点: "]</strong>[diagnose.choices[diagnosis].why]</p>
144     </fieldset>
145   </div>
146   <div className={styles.workshopStep}>
147     <div className={styles.stepTitle}><span>03</span><h3>搭建: 为每段挑一句能回答问题的话</h3></div>
148     <p className={styles.question}>题目: 请讲一次你和同学协作完成任务的经历.</p>
149     {starParts.map((part) => <fieldset key={part.key} className={styles.exercise}><legend>[part.letter] · {part.title}: {part.prompt}</legend>
150       {[["good", "weak"] as const}.map((kind) => <label key={kind} className={styles.choice}>
151         <input type="radio" name={`build-${part.key}`} checked={choices[part.key] === kind} onChange={() => setChoices((prev) => ({ ...prev, [part.key]: kind }))) />
152         <span>{buildChoices[part.key][kind]}</span>
153       </label>)}
154       {choices[part.key] && <p className={styles.explanation} role="status">{buildChoices[part.key].why}</p>
155       </fieldset>)}
156     {assembled && <div className={styles.preview}><h4>面试官听到的连贯回答</h4><p>{assembled}</p></div>
157     </div>
158   </div>
159   <div className={styles.workshopStep}>
160     <div className={styles.stepTitle}><span>04</span><h3>轮到你: 用一段真实经历试着写</h3></div>
161     <p className={styles.question}>可以选择课程作业、社团、志愿或实习。只写确实发生过的事, 尽量突出本人行动.</p>
162     <div className={styles.draftGrid}>{starParts.map((part) => <label key={part.key} className={styles.draftField}>
163       <span><strong>[part.letter] · {part.title}</strong> {part.prompt}</span>
164       <textarea value={draft[part.key]} rows={4} onChange={(event) => { setDraft((prev) => ({ ...prev, [part.key]: event.target.value })); setReviewed(false); }} placeholder="写下真实细节; 不需要编造数字" />
165       </label>)}</div>
166     <div className={styles.reviewActions}><button type="button" onClick={() => setReviewed(true)}>检查结构</button>
167     <button type="button" className={styles.secondary} onClick={() => { setDraft(emptyDraft); setReviewed(false); }}>清空重写</button></div>
168     {reviewed && <p className={styles.explanation} role="status">[missing.length ? `还可以补充: ${starParts.filter((p) => missing.includes(p.key)).map((p) => p.title).join(", ")}` : "四段都有内容。请再核对是否真实、是否回答了题目, 以及行动和结果是否具体。填写完整不代表获得面试分数。"]</p>
169     {built && <div className={styles.preview}><h4>你的答案草稿</h4><p>{built}</p></div>
170     </div>
171   </section>
```

10 逐题反馈隔离

证明仅新生本人可读取最新答案的练习反馈；它不推进会话，也不写正式报告。

反馈路由角色依赖 [独立逻辑块]

server/api/answer_feedback.py / 行 10-14

```
10 router = APIRouter(  
11     prefix="/api/sessions",  
12     tags=["answer-feedback"],  
13     dependencies=[Depends(require_student)],  
14 )
```

逐题反馈接口

server/api/answer_feedback.py / 行 18-67

```
18 async def get_answer_feedback(request: Request, sid: int = Path(..., ge=1)):  
19     _ensure_student_session_access(request, sid)  
20  
21     with SessionLocal() as db:  
22         session = db.get(Session, sid)  
23         if session is None:  
24             raise HTTPException(  
25                 status_code=404,  
26                 detail={"code": "SESSION_NOT_FOUND", "message": "面试会话不存在"},  
27             )  
28         if session.mode != "新生":  
29             raise HTTPException(  
30                 status_code=403,  
31                 detail={"code": "MODE_FORBIDDEN", "message": "即时练习反馈仅适用于新生模式"},  
32             )  
33         if session.status == "answering":  
34             raise HTTPException(  
35                 status_code=409,  
36                 detail={"code": "ANSWER_IN_PROGRESS", "message": "回答仍在处理中"},  
37             )  
38  
39         answer = (  
40             db.query(Answer)  
41             .filter(Answer.session_id == sid)  
42             .order_by(Answer.id.desc())  
43             .first()  
44         )  
45         if answer is None:  
46             raise HTTPException(  
47                 status_code=409,  
48                 detail={"code": "ANSWER_NOT_FOUND", "message": "请先完成本题回答"},  
49             )  
50         answer_id = answer.id  
51         q_seq = answer.q_seq  
52         is_followup = answer.is_followup  
53         question_text = answer.question_text  
54         answer_text = answer.answer_text  
55  
56         feedback = await evaluate_answer(  
57             question_text=question_text,  
58             answer_text=answer_text,  
59             is_followup=is_followup,  
60         )  
61         return AnswerFeedbackResponse(  
62             answer_id=answer_id,  
63             q_seq=q_seq,  
64             is_followup=is_followup,  
65             question_text=question_text,
```

逐题反馈接口 / 续 / server/api/answer_feedback.py

```
66         feedback=feedback,  
67     )
```

模型片段扩展为展示原句

server/services/answer_feedback.py / 行 46-63

```
46 def _complete_evidence_quote(answer_text: str, fragment: str | None) -> str | None:  
47     """Expand a verified fragment to its containing original sentence."""  
48     if not fragment:  
49         return None  
50     fragment = fragment.strip()  
51     if not fragment:  
52         return None  
53     hit = answer_text.find(fragment)  
54     if hit < 0:  
55         return None  
56  
57     sentence_marks = "。！？\n"  
58     start = max(answer_text.rfind(mark, 0, hit) for mark in sentence_marks) + 1  
59     ends = [answer_text.find(mark, hit + len(fragment) - 1) for mark in sentence_marks]  
60     ends = [position for position in ends if position >= 0]  
61     end = min(ends) + 1 if ends else len(answer_text)  
62     quote = answer_text[start:end].strip().strip('“”"“”')  
63     return quote if quote and quote in answer_text else fragment
```

练习评分与回退

server/services/answer_feedback.py / 行 111-146

```
111 async def evaluate_answer(  
112     *, question_text: str, answer_text: str, is_followup: bool  
113 ) -> AnswerFeedbackResult:  
114     """Return an ephemeral practice assessment; it is never written to reports."""  
115     try:  
116         result = await chat_json(  
117             "answer_feedback",  
118             [  
119                 {"role": "system", "content": _SYSTEM_PROMPT},  
120                 {  
121                     "role": "user",  
122                     "content": json.dumps(  
123                         {  
124                             "question": question_text,  
125                             "answer": answer_text,  
126                             "is_followup": is_followup,  
127                             "quote_candidate": answer_text.strip()[:25],  
128                         },  
129                     ensure_ascii=False,  
130                 ),  
131                 ],  
132             ),  
133             AnswerFeedbackLlmResponse,  
134         )  
135     except Exception:  
136         return _fallback(question_text, answer_text, is_followup)  
137  
138     quote = _complete_evidence_quote(answer_text, result.evidence_quote)  
139     reliable_quote = bool(quote and quote in answer_text)  
140     if result.practice_score is None or not reliable_quote:  
141         return _fallback(question_text, answer_text, is_followup)  
142     return AnswerFeedbackResult(  
143         **result.model_dump(exclude={"evidence_quote"}),  
144         evidence_quote=quote,  
145         basis="ai",  
146     )
```

答辩说明与实现边界

一条可演示的链

学生登录后选择岗位，六题面试生成报告；同一报告进入成长记录与企业初筛。新生咨询可引向这两个岗位。

评分证据的边界

正式评分只校验 `evidence` 是回答的连续原文、长度不超过 25 字；这不保证模型评价在语义上正确。

逐题反馈的展示原句

反馈模型给出最多 25 字片段；程序扩展成原回答中的整句供展示。练习反馈与正式报告分离。

声学数值的边界

语速、停顿和填充词计算可复现；尚不能据此宣称其测量效度、就业效果或能力提升。

并发与重试

租约防止并发写入；最终提交须持有 `token`。网络结果不明时先查询状态，不能把 409 解释为自动安全重投。

模型用量指标

当前非流式调用的 `ttft_ms` 记录响应可用耗时，不是真正的首 `token` 时延。

成长口径

综合分跨输入模式不直接比较；同评分版本且输入模式均已知时，共同有效的非声学维度可比较。缺失值保留为空。

企业排序的性质

默认旧配置把面试、学历、学校权重分为 60%/20%/20%；可配置，仅供初筛偏好，不能等同面试能力，需人工复核。